

Claude Code in Action

AI-Assisted Development from Idea to Deployment

Sriman Abimanyu M

Founder & CEO, SymphoZen Labs

Full-Day Hands-On Workshop

One day. One engineering journey.

SESSION 1

01

Foundations

Software evolution,
sequence models,
LLMs, prompts

SESSION 2

02

Engineering

Context, agents,
specifications,
harnesses, loops

SESSION 3

03

Ecosystem

MCP, tools, alignment,
responsible AI

SESSION 4

04

Build & ship

CampusHub, mini-
project, deployment,
Q&A

Today's **challenge**

Can we build and deploy a useful application before 5 PM?

Specify it. Build it. Test it. Review it. Ship it.

How developers traditionally **worked**



The developer manually coordinates every stage.

From suggestions to **action**

- 01 Manual coding
- ↓
- 02 IDE autocomplete
- ↓
- 03 AI code completion
- ↓
- 04 Chat-based generation
- ↓
- 05 Repository-aware agents
- ↓
- 06 **Agentic workflows**

The important shift: AI that suggests → AI that acts.

Chatbot. Assistant. **Agent.**

01

Chatbot

Answers a question

User provides context

User executes the answer

02

Coding assistant

Suggests code

Works with limited context

User directs the changes

03

Coding agent

Completes a task

Explores repository context

Edits, executes, checks, iterates

The developer is still **responsible**

AI does not remove engineering responsibility.

It increases the importance of good engineering decisions.

The workshop **journey**



SECTION 2

Understand

RNN → LSTM → Attention → Transformer → LLM → Agent

Where models and agents **fit**

01 Artificial Intelligence

02 Machine Learning

03 Deep Learning

04 Generative AI

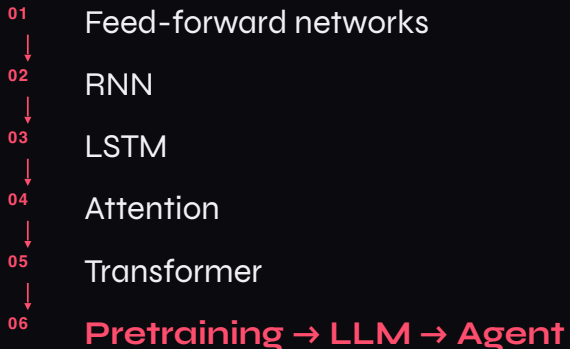
05 Large Language Models

Model + context + tools
+ permissions +
feedback

AI Agents

Agents combine models with context, tools, permissions and feedback.

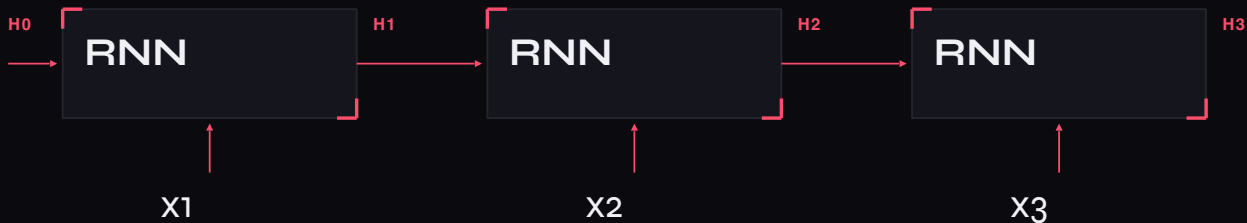
Each generation solved a **limitation**



A simplified history, not a claim that each architecture replaced every earlier one.

RNN: carry the past forward

A hidden state carries information from earlier steps.



Shared recurrent cell, unrolled over three steps

Remember something about the past while processing the present.

When the important word is far away

The student who submitted several assignments after weeks of work was exhausted.

STUDENT

WAS

Vanishing / exploding
gradients

Long-range
dependencies

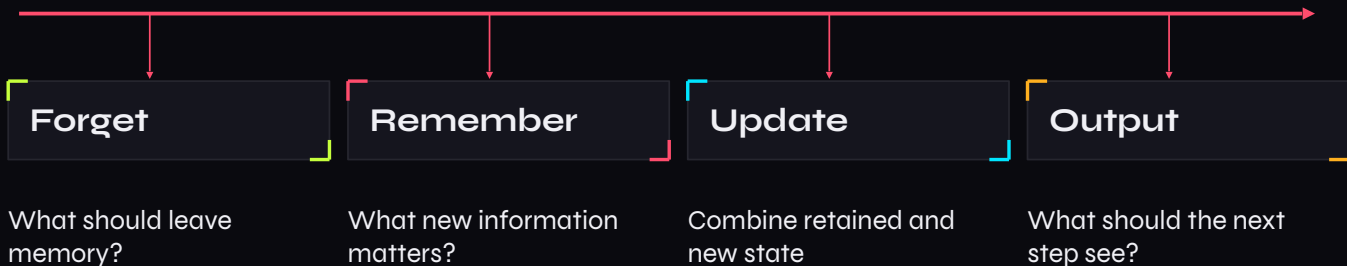
Sequential computation

Information and training gradients must travel through many recurrent steps.

LSTM: a memory path with **gates**

HOCHREITER & SCHMIDHUBER, 1997

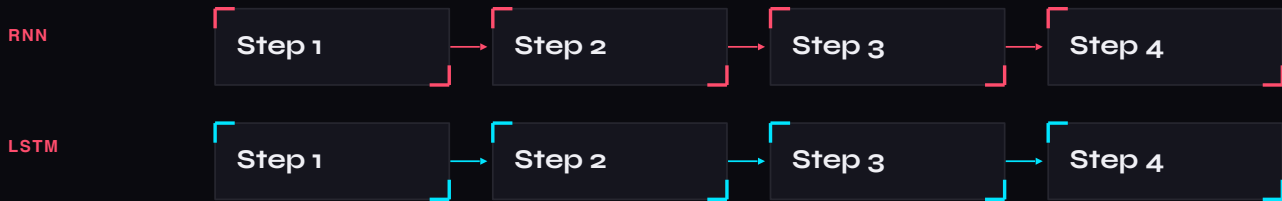
CELL STATE: CARRY SELECTED INFORMATION FORWARD



A memory cell and gates improve long-term dependency learning.

Better memory. Still **sequential**.

LSTM improved memory, but processing still advances one step at a time.

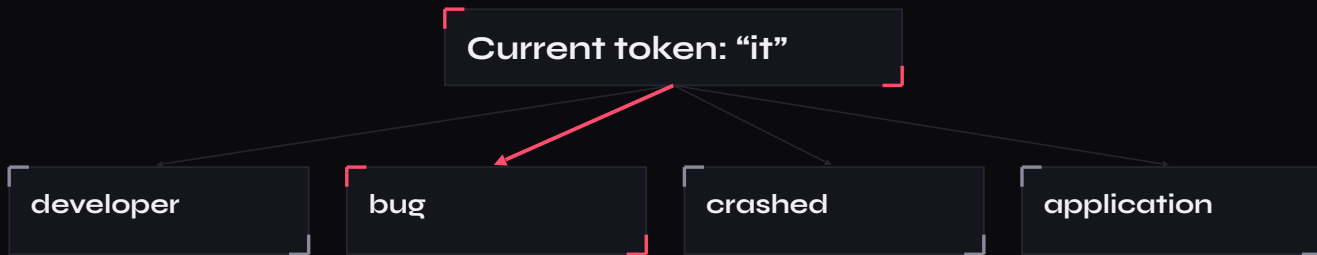


Limited parallelization / Long training times / Long paths between distant tokens

We need a more direct relationship between tokens.

What matters to this token?

The developer fixed the bug because it crashed the application.



Illustrative relevance links, not attention weights measured from a model.

Attention Is All You **Need**

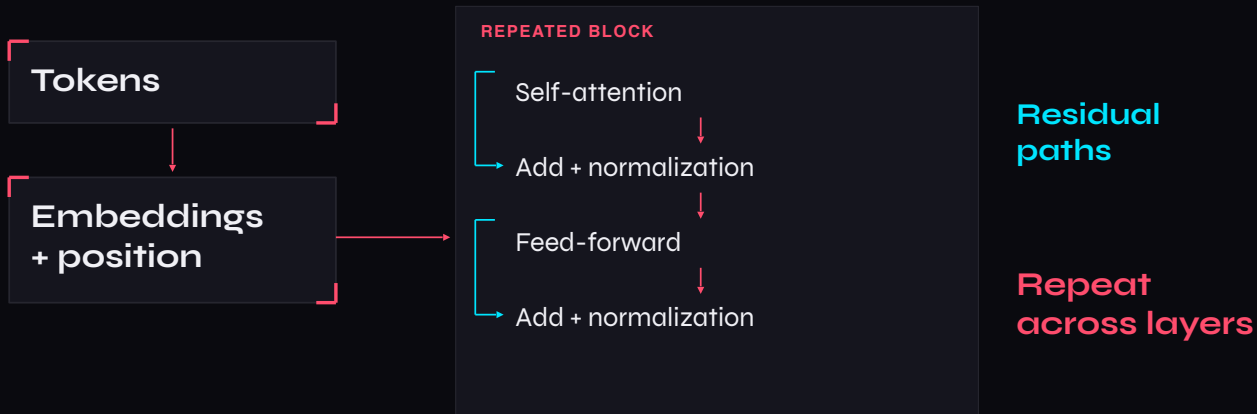
2017

Vaswani et al. · 2017

- Attention existed before 2017.
- The Transformer removed recurrence and convolution from its core sequence architecture.
- More parallel training. Shorter information paths. Effective scaling.

The breakthrough was the architecture, not the invention of attention.

Inside a Transformer **block**



Original Transformer: encoder-decoder. Many generative LLMs are decoder-oriented.

Query. Key. Value.

01

Query

What am I looking for?

02

Key

What information do I represent?

03

Value

What information should I contribute?

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

In a causal decoder, a token can attend only to positions allowed by the attention mask.

Attention matters for code

CAMPUSHUB / EVENT MATCHING

```
type CampusEvent = {  
  title: string; category: string;  
};  
  
const matches =  
  (event: CampusEvent, query: string) =>  
    event.title.toLowerCase()  
      .includes(query.toLowerCase());
```

Type



Parameter

Declarations, usages, imports,
interfaces and tests carry
relationships across code.

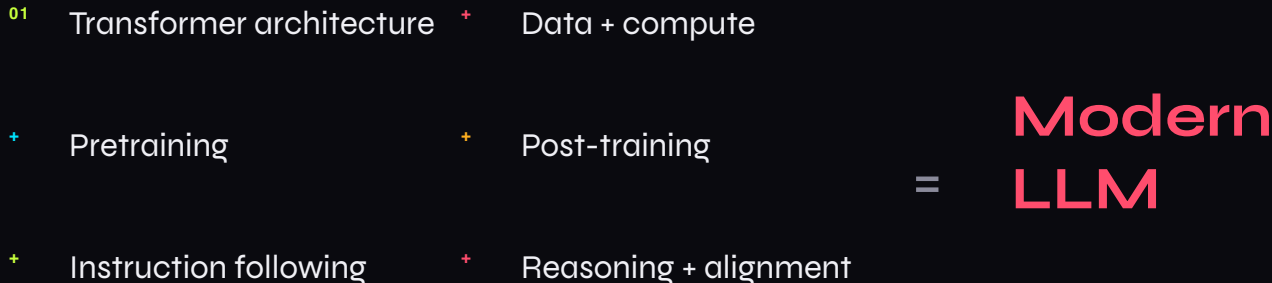
Only files and tokens actually available in context can inform the current response.

RNN / LSTM vs Transformer

RNN / LSTM	Transformer
Sequential processing	Highly parallelizable training
Long information paths	Direct token-to-token attention
Hidden-state bottleneck	Rich contextual representations
Difficult scaling	Effective scaling with data and compute
Long-range challenges	Stronger long-context modelling

Parallel training does not mean autoregressive generation predicts all tokens at once.

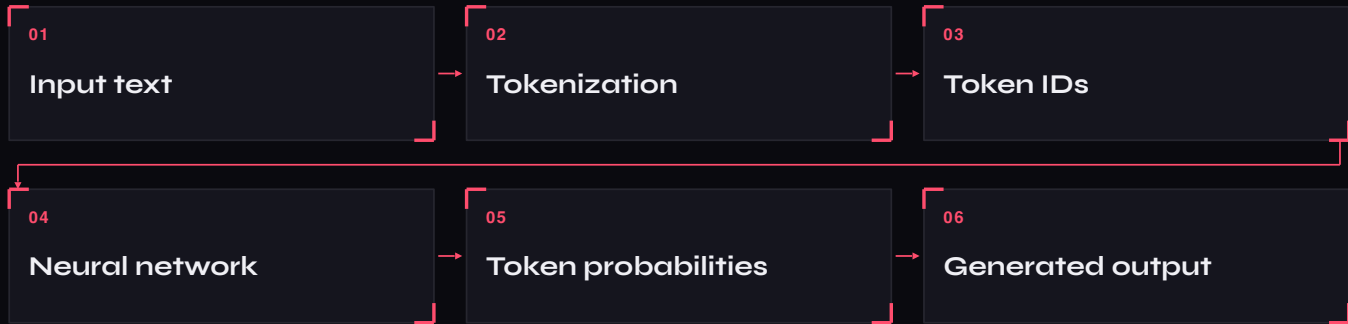
A Transformer is not yet an LLM



Architecture is one ingredient in a larger training system.

Text becomes numbers, then **text**

A language model predicts useful sequences of tokens.



Tokenization

Claude Code can build applications.



Word ▪ part of a word ▪ punctuation ▪ code symbol ▪ whitespace

Illustrative segmentation: actual token boundaries depend on the tokenizer.

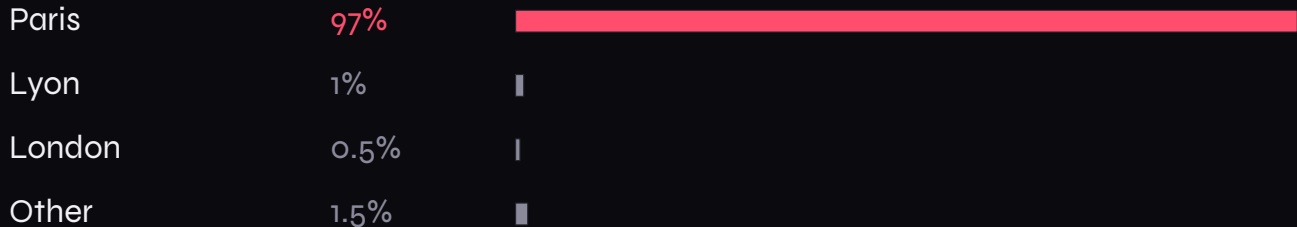
Tokens are a working **budget**

Relevant context is more valuable than excessive context.

- Context-window usage
- Cost and response length
- Processing time
- Code the model can inspect
- Conversation available at once

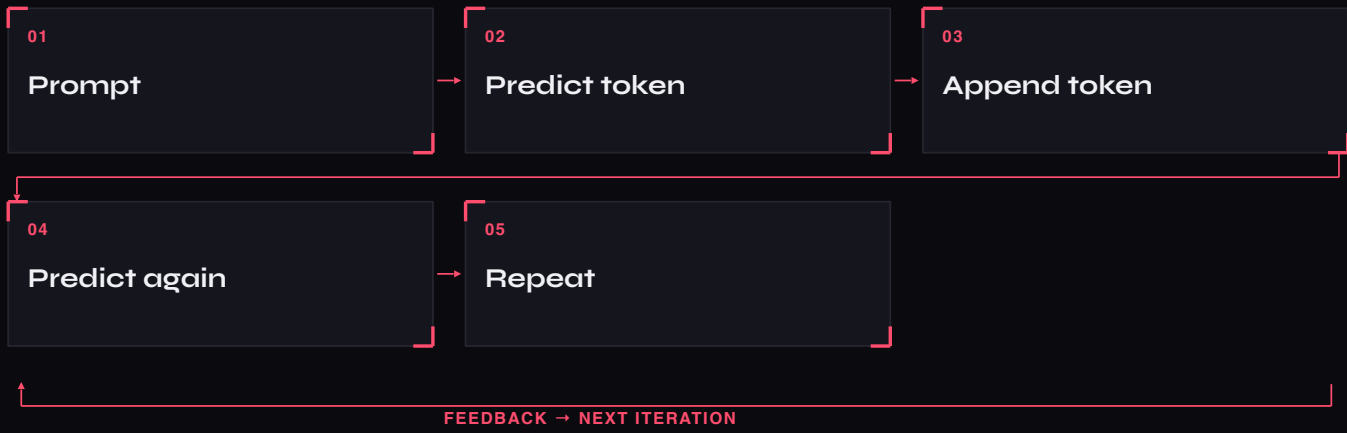
Predicting the next token

The capital of France is...



Illustrative probabilities, not measured model output.

Generation happens token by **token**



Learned relationships connect language, concepts, code and reasoning patterns.

Why answers **differ**

AI responses are probabilistic.

- Wording and system instructions
- Available files and documentation
- Sampling choices
- Tool results
- Conversation history

Plausible is not guaranteed **true**

Generated output can be wrong,
insecure, outdated or incompatible.

**Grounding + tools + tests + review +
verification**

An LLM is not a coding agent

$$\begin{array}{lcl}
 \text{01} & \text{LLM + repository context} & + \quad \text{Terminal + file editing} \\
 & + \quad \text{Git + tests} & + \quad \text{MCP + external tools} \\
 & + \quad \text{State + permissions} & + \quad \text{Feedback loop + human approval}
 \end{array}
 = \text{AI Coding Agent}$$

Coding performance is a **system**

**Model capability × context quality × tool reliability
× agent loop × verification**

- Prompt and context selection
- Repository and tool access
- Iterations and stop conditions
- Test environment and runtime budget

A conceptual relationship, not a calibrated mathematical formula.

Claude's coding **strengths**

**The model matters.
The harness matters
too.**

- Code understanding and instruction following
- Long-context repository reasoning
- Multi-step planning and tool use
- Execution and root-cause debugging
- Iterative verification

Public evidence does not establish that Anthropic uses a fundamentally superior Transformer architecture.

Read coding benchmarks in **context**

ANTHROPIC-REPORTED RESULTS · JULY 2026

Frontier-Bench v0.1

Opus 5 leads in Anthropic's published evaluation.

CursorBench 3.2

Strong results; effort and cost settings matter.

Long-running coding

Reported improvement in verification and iteration.

A benchmark result is not a universal ranking across all models, tasks and setups.

SECTION 3

Prompt

What should I tell the AI?

An ambiguous starting point

CAMPUSHUB / PROMPT

Build a campus event application.

Users? / Features? / Stack? / Authentication? / Success criteria?

Ambiguity creates inconsistent results.

Anatomy of a strong prompt

01 Role

02 Goal

03 Context

04 Constraints

05 Input

06 Expected output

07 Success criteria

08 Verification

Weak → better → engineering-grade

WEAK

Build a login page.

BETTER

Build the CampusHub login page using the existing SvelteKit, TypeScript and Tailwind components.

ENGINEERING-GRADE

Reuse authentication. Validate email and password. Include loading and server-error states, accessible labels and keyboard support. Run type checking.

Each refinement replaces guesses with requirements.

Context beats **flattery**

WEAK

Act as the world's greatest developer and build this perfectly.

BETTER

CampusHub uses SvelteKit, TypeScript, Tailwind CSS, Firebase Authentication and Firestore. Reuse the existing session-management implementation.

Do not create an additional authentication system.

Constraints narrow the solution **space**

- Preserve authentication and public API contracts
- Use existing components and naming conventions
- Avoid unnecessary dependencies
- Preserve backward compatibility
- Run tests after implementation
- Explain architectural changes before making them

Inspect before acting

CAMPUSHUB / PROMPT

Inspect CampusHub's routing, authentication, data-access layer and reusable components. Summarize the architecture and propose an implementation plan. Do not modify files yet.

Understanding comes before execution.

Show what good looks like

CAMPUSHUB / PROMPT

Here is an existing form component. Build the CampusHub event-submission form using the same structure, validation pattern, naming conventions and error handling.

[Code style](#) / [UI patterns](#) / [API shapes](#) / [Tests](#) / [Documentation](#)

Few-shot prompting gives the model an example to follow.

Make assumptions and verification explicit

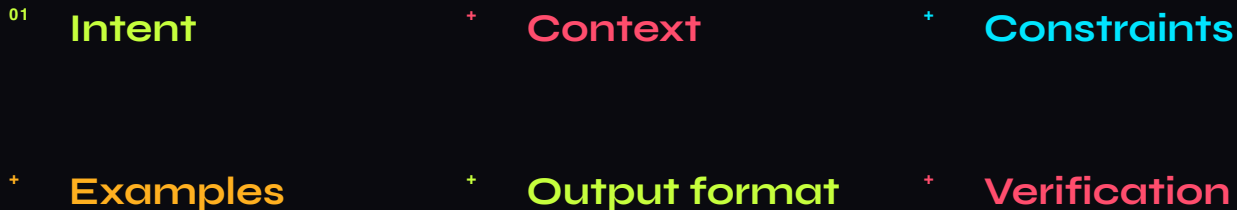
BEFORE IMPLEMENTATION

List assumptions that affect architecture, security, data modelling or user experience.

AFTER IMPLEMENTATION

Type-check, test and build. Verify acceptance criteria, report remaining warnings and summarize changed files.

A prompt is structured **thinking**



Prompt vs context **engineering**

PROMPT ENGINEERING

What should I ask?

CONTEXT ENGINEERING

What should the AI know
before attempting the task?

A clear instruction works best inside a useful information environment.

The context **window**

The model can work only with information available in context.

Instructions + CLAUDE.md

Architecture + repository files

Issues + schemas + API docs

Design references

Conversation + tool results

Current task

Context is limited. Include what is necessary.

Curate the **context**

Too little

Guessing, duplicated work, invented APIs

Too much

Noise, conflicts, irrelevant files

Relevant

Current, accurate, structured, retrievable

More context is not automatically better context.

CLAUDE.md

Persistent project guidance reduces repeated prompting.

CAMPUSHUB / CLAUDE.MD

- # Project overview + stack
- # Architecture + repository structure
- # Development commands
- # Coding conventions + Git workflow
- # Testing + security constraints
- # Definition of done

Instructions guide behavior; enforced permissions and hooks provide stronger controls.

The context **hierarchy**

01 Organization rules

02 Project rules

03 CLAUDE.md

04 Architecture

05 Feature specification

06 Relevant repository files

07 **Current task**

The new developer **skill**

Design the environment in which AI can produce reliable code.

Instructions, context, specifications, tools, permissions, tests and feedback.

SECTION 5

Collaborate

Claude Code is a repository-aware development agent.

The agent loop



Iterative execution continues until a completion condition is met.

Read the repository **first**

Repository awareness distinguishes an agent from snippet generation.

- Package and build configuration
- Routes and reusable components
- Data access and authentication
- Environment and deployment settings
- Existing tests

Tool calling turns language into **action**

Read + search files

Edit files

Terminal + tests

Git + issue tracker

Browser + database

MCP + external
services

Confidence meets **evidence**

LANGUAGE MODEL

This code should compile.

CODING AGENT

I will run the compiler and inspect the actual result.

Tools allow the agent to replace assumptions with evidence.

Permissions and human **control**

READ

Files and systems the agent may inspect

MODIFY

Code and data the agent may change

APPROVE

Sensitive commands and production actions

OFF-LIMITS

Secrets and protected infrastructure

Plan before implementation



A reviewed plan reduces accidental architectural damage.

Review the **diff**

REVIEW / TERMINAL

```
git status  
git diff
```

- Unexpected files and unrelated changes
- New dependencies
- Security-sensitive changes
- Code quality and conventions
- Missing tests or documentation

The developer owns the review.

Stop Prompting. Start **Specifying**.

A one-line prompt describes a desire.

A specification describes the system to be built.

Anatomy of a specification

PURPOSE

Problem, users, goals, scope

SYSTEM

Requirements, architecture, data model

BEHAVIOR

User flows, UI states, validation, errors

PROOF

Edge cases, acceptance criteria

CampusHub: event search

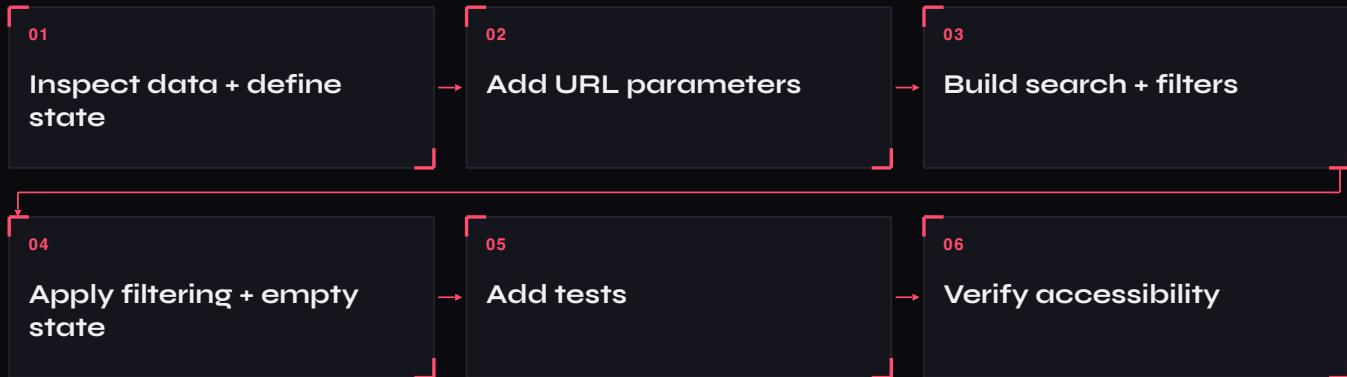
As a student, I want to search and filter campus events so I can find events relevant to me.

- Search event titles
- Filter by category and date
- Preserve filters in the URL
- Show an empty state
- Work on mobile and desktop

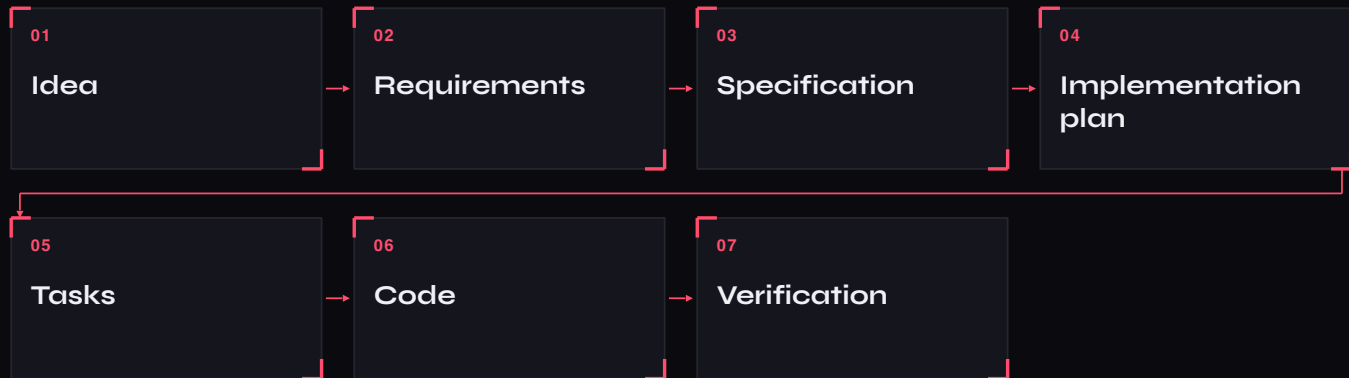
Acceptance criteria make it **testable**

- 01 Search updates the event list
- 02 Multiple filters work together
- 03 Clearing filters restores all events
- 04 No console errors
- 05 Keyboard navigation remains accessible

Turn the specification into **tasks**



The spec-driven lifecycle



Define done before building

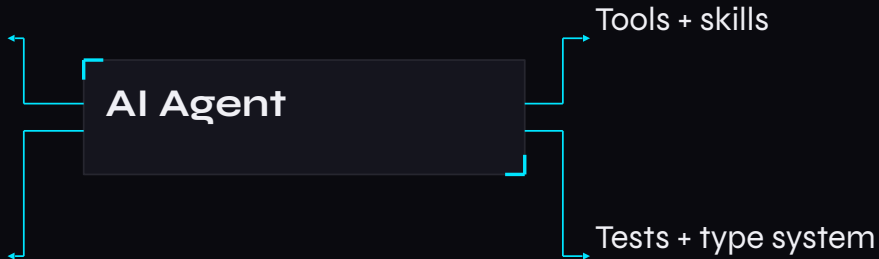
- 01 Requirements implemented; code reviewed
- 02 TypeScript and production build pass
- 03 Tests pass; no console errors
- 04 Responsive and accessible flows verified
- 05 Documentation updated

The harness surrounds the **agent**

Prompt engineering improves the instruction. Harness engineering improves the surrounding system.

Rules + context

Permissions + hooks



CI/CD ▪ linters ▪ subagents ▪ observability ▪ feedback

Weak harness vs strong harness

WEAK

Generate the feature and make sure it works.

- 01 Generate
- 02 Format + type-check
- 03 Test + build
- 04 Verify UI + security
- 05 Fix failures

Deterministic guardrails

AI is probabilistic. Build a system that detects when it is wrong.

Types, compilers, schemas, validation, tests, lint, CI, permissions, hooks and deployment checks.

Human review **gates**

- Architecture changes
- Database migrations and authentication
- Security-sensitive and billing logic
- Production deployment
- Destructive operations

Automation reduces effort. Accountability remains with people.

Build. Observe. Improve.



Reliable feedback until clearly defined stop conditions are satisfied.

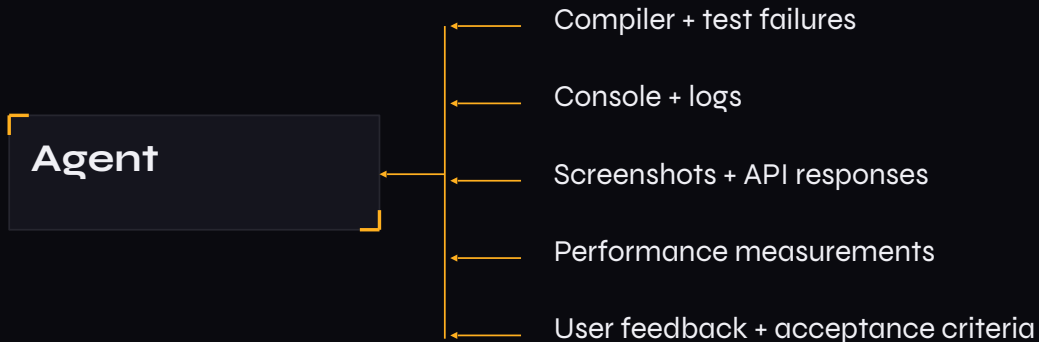
A reliable debugging loop

WEAK

Fix the
application.

- 01 Run failing test
- 02 Identify root cause
- 03 Change relevant code
- 04 Rerun test + full suite
- 05 Report remaining failures

Give the agent useful **feedback**



Without feedback, the agent can only guess.

Stop when the criteria are **satisfied**

- 01 Required functionality works
- 02 Tests and build pass
- 03 No unresolved errors
- 04 Acceptance criteria satisfied
- 05 No unauthorized files changed

Five engineering **layers**

Prompt engineering

What should I ask?

Context engineering

What should the AI know?

Spec-driven development

What exactly are we building?

Harness engineering

What tools, rules and checks surround the AI?

Loop engineering

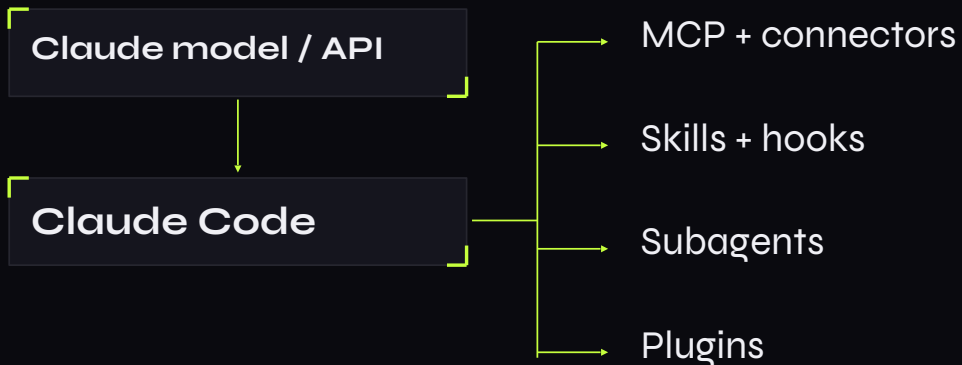
How does it verify and improve?

SECTION 9

Connect

Models become useful inside working systems.

Claude is more than a **model**



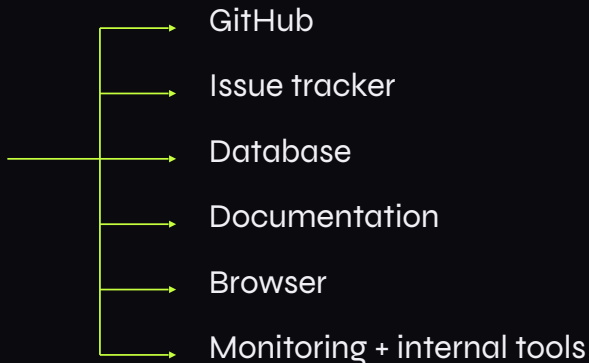
MCP: a common connection protocol

Claude Code



MCP

Like USB-C for AI tools.



A standard interface connects AI applications to external tools and data.

Connectors change the **task**

WEAK

Tell me how to solve issue 128.

BETTER

Read issue 128, inspect the repository, implement the fix, run tests and prepare a summary.

Skills make procedures **reusable**

Custom procedures Claude can load when needed.

`/code-review`

`/write-tests`

`/security-audit`

`/update-documentation`

`/create-api`

`/deploy-preview`

Example custom skill names, not a list of built-in commands.

Hooks act at workflow **events**

01

Before tool use

Block dangerous commands or protected files

02

After edits

Run formatting or targeted checks

03

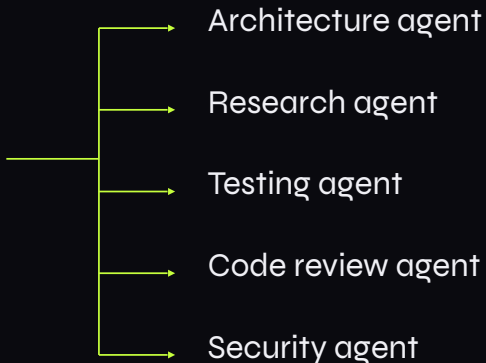
Before completion

Run configured verification

Hook events and handlers must be configured; checks do not run automatically just because they appear here.

Delegate focused **work**

**Main
Development
Agent**



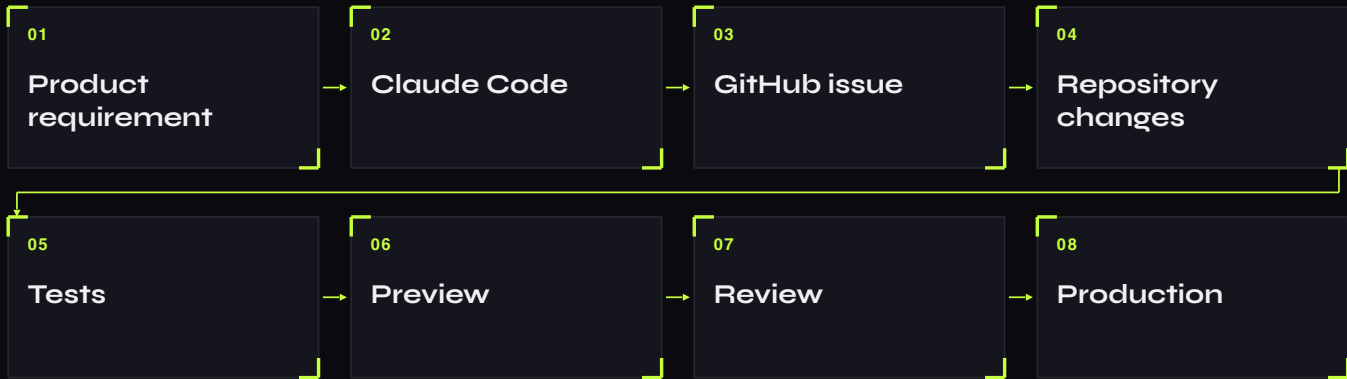
Focused responsibilities and separate context return useful results to the main agent.

Plugins package **capabilities**

**A reusable
development
toolkit.**

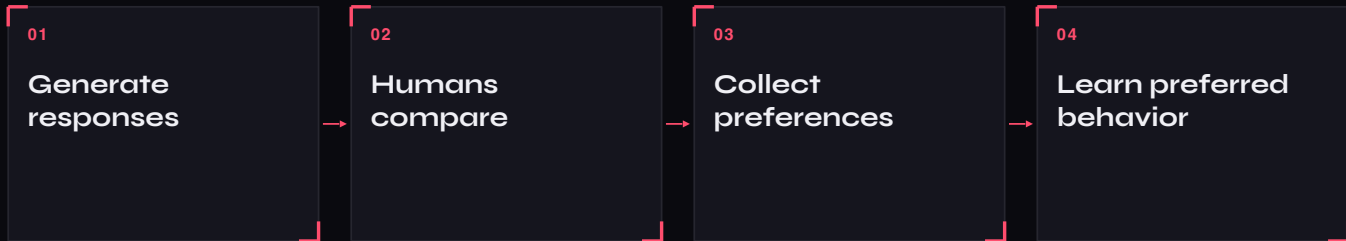
- Commands + skills
- Hooks
- Subagents
- MCP integrations
- Project workflows

From requirement to connected workflow



RLHF: human preference

Alignment aims for more helpful, safer, more consistent behavior.



Principles guide critique and revision

Constitutional AI



Why developers should care

- Refuse unsafe actions
- Ask for confirmation
- Warn about risky commands
- Avoid exposing secrets
- Limit sensitive operations
- Challenge harmful instructions

These behaviors are part of responsible agent design.

AI-generated does not mean **ready**

AI-GENERATED ≠ CORRECT

AI-GENERATED ≠ SECURE

AI-GENERATED ≠ PRODUCTION-READY

You own the code you **ship**.

Security + privacy

Correctness +
accessibility

Performance +
testing

Licensing +
dependencies

Data protection

Deployment
decisions

Blind acceptance creates real **risk**

Understand the code before accepting it.

- Insecure code, exposed API keys and suspicious packages
- License issues and incorrect migrations
- Outdated APIs and unauthorized code sharing
- Deployment without tests
- Losing understanding of the codebase

Use AI to learn **faster**

Do not become a developer who can only code with AI.

Become a developer who understands software and can build better because of AI.

LIVE BUILD

CampusHub

A campus event discovery application.

Browse + search + filter / Event details + bookmarks / Event submissions / Simple dashboard

CampusHub: idea to deployment



Start with the idea

Build a campus event application where students can discover and submit events.

- 01 Identify users, workflows and missing requirements
- 02 Expose risks and assumptions
- 03 Review scope and acceptance criteria

CHECKPOINT 01 / REQUIREMENTS AGREED

Create the **specification**

Define behavior before code.

- 01 Routes, features and data model
- 02 UI states, validation and errors
- 03 Accessibility, tests and deployment expectations

CHECKPOINT 02 / SPECIFICATION REVIEWED

Inspect the starter. Establish **context**.

Repository



CLAUDE.md

No code changes yet.

- 01 Read configuration, routes and existing components
- 02 Find tests and deployment settings
- 03 Create CLAUDE.md with conventions and definition of done

CHECKPOINT 03 / CONTEXT READY

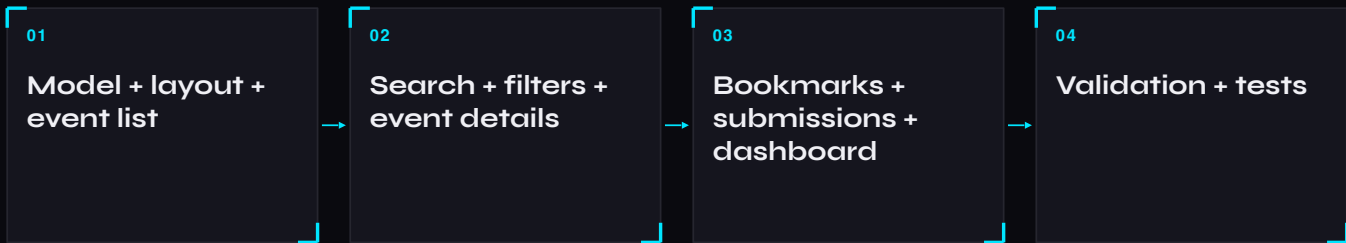
Review the implementation plan

Make the sequence reviewable.

- 01 Ordered tasks and dependencies
- 02 Files, data flow and risk areas
- 03 Test strategy and deployment steps

Implement incrementally

Watch the agent read, edit, execute and respond to feedback.



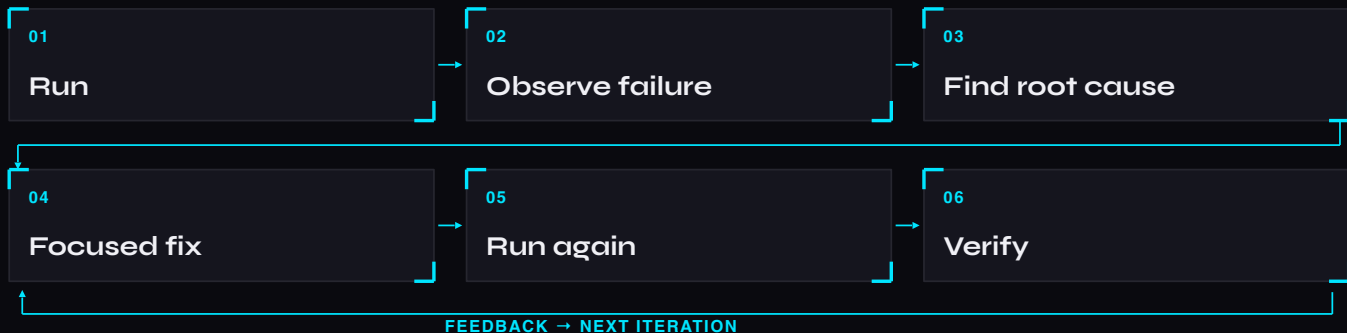
Deliberate failure

CampusHub search and category filters do not work together.

- 01 Reproduce the mismatch
- 02 Inspect the actual result
- 03 State the expected behavior

Diagnose before patching

Reproduce first. Explain the root cause. Make the smallest safe change. Rerun the relevant test and full verification suite.



CHECKPOINT 06 / FAILURE REPRODUCED AND RESOLVED

Verify code and browser behavior

A green build is one checkpoint, not the whole definition of done.

01 Type checks, tests and production build

02 Search, filters, forms and empty states

03 Mobile layout, keyboard use and console

04 Acceptance criteria and unresolved warnings

Review the CampusHub **diff**

REVIEW / TERMINAL

```
git status  
git diff
```

- Unexpected files, dependencies or large changes
- Secrets and security-sensitive modifications
- Formatting noise, TODOs and missing documentation

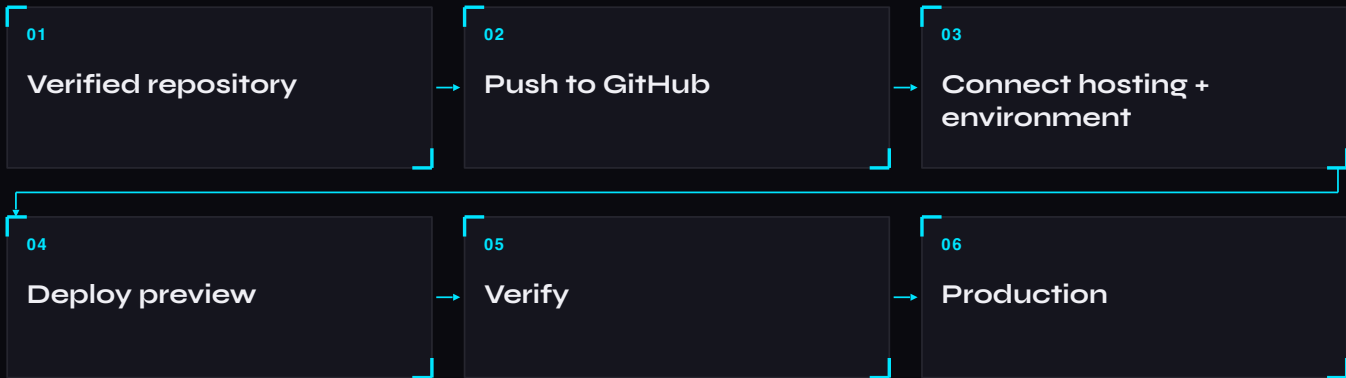
Document and commit

feat: add campus event discovery and filtering

- 01 Update setup, environment and available scripts
- 02 Document architecture, features, testing and deployment
- 03 Commit focused, reviewed, verified changes without secrets

CHECKPOINT 09 / REVIEWABLE COMMIT

Deploy verified code



The reveal

At 10:00 AM, this was only an idea.

Now it is specified, implemented,
tested, documented, version-
controlled and deployed.

SECTION 12

Your turn

Demonstrate the engineering workflow.

One focused **feature**

Work from a starter project.

- 01 Student Expense Tracker
- 02 Campus Lost and Found
- 03 Study Planner
- 04 Event Registration App
- 05 Placement Preparation Tracker

Choose search, a form, bookmarking, validation, a dashboard summary or tests.

Understand → specify → plan → **build**

Use existing conventions. Be ready to explain the code.

01



02



03



04

Inspect the starter and
write a short spec

Review the plan before
implementing

Run verification and
review the diff

Stop at acceptance
criteria; prepare a
short demo

Evaluate the workflow, not the amount of generated code.

What we will look **for**

Understanding

Correct interpretation

Specification

Scope and acceptance criteria

Planning

Logical sequence

Code quality

Readable and consistent

Verification

Tests, checks or manual validation

Review + explanation

Understands the changes and result

Team demo: show your **engineering**

- What problem did you solve?
- What did you ask Claude to do?
- What context did you provide?
- What went wrong?
- How did you verify the result?
- What would you improve next?

AI in professional **teams**

AGENTS ASSIST

Features, bugs, tests, docs, reviews, migrations, triage, releases, internal tools and prototypes

HUMANS OWN

Product, users, trade-offs, architecture, ethics, security, quality, approval and accountability

Professional work keeps review **gates**



Monitoring and feedback feed the next engineering cycle.

The AI-native developer **stack**

Idea → Agent → Working software

Prompt engineering

What do I ask the AI?

Context engineering

What should the AI know?

Spec-driven development

What exactly are we building?

Harness engineering

What tools and guardrails surround it?

Loop engineering

How does it verify and improve?

What you learned **today**

Models

Tokens, generation and hallucinations

Instructions

Prompts, context and specifications

Agents

Claude Code, tool calling and MCP

Reliability

Harnesses, loops, testing and review

Delivery

Idea to deployment

Good engineering matters **more**

AI does not eliminate engineering.

**It makes good engineering more
important.**

Questions?

- Junior developers and careers
- Learning to code with AI
- Security and private repositories
- Complete applications and agent mistakes

Ask about anything from today's demo.

Research foundations

HOCHREITER & SCHMIDHUBER, 1997

[Long Short-Term Memory](#)

BAHDANAU, CHO & BENGIO, 2014

[Neural Machine Translation by Jointly Learning to Align and Translate](#)

VASWANI ET AL., 2017

[Attention Is All You Need](#)

ANTHROPIC

[Constitutional AI: Harmlessness from AI Feedback](#)

Engineering references & resources

ANTHROPIC ENGINEERING

[SWE-bench: model and scaffold](#)

ANTHROPIC, FEBRUARY 2026

[Infrastructure noise in agentic coding evaluations](#)

ANTHROPIC, JULY 2026

[Introducing Claude Opus 5](#)

CLAUDE CODE DOCUMENTATION

[Memory, skills, hooks, subagents and plugins](#)

MODEL CONTEXT PROTOCOL

[Tools and data interoperability](#)

Thank You

Claude Code in Action

From Idea to Specification to Code to Production

Sriman Abimanyu M
Founder & CEO, SymphoZen Labs

AI can generate code. Engineers create reliable software.



Scan & Lets Connect



Scan & Lets Connect

Your Feedback Matters

